



Universita Politecnica delle Marche

Corso di Ingegneria del Software

TicketSeller

Documento di progetto

Analisi, progettazione, implementazione e testing

Docente	Domenico Ursino
Studenti	Kullolli Aaron Valerio Mancigotti Umberto D'Urso

Anno Accademico 2025–2026

Indice

1	Descrizione del progetto	1
1.1	Obiettivo generale	1
1.2	Scelte architetturali	1
1.3	Architettura generale	1
2	Analisi dei requisiti	2
2.1	Criteri di qualita	2
2.2	Attori	3
2.3	Requisiti funzionali	3
2.4	Requisiti non funzionali	3
3	Diagramma dei casi d'uso	4
3.1	Casi d'uso principali	5
3.2	Caso d'uso UC3	5
4	Matrice di mapping	5
5	Package di analisi	6
5.1	Package logici	6
5.2	Package implementativi	7
6	Diagramma di sequenza	8
6.1	Scenario UC3	8
6.2	Scenario UC5	9
7	Diagrammi di attivita	10
7.1	Acquisto biglietto	10
7.2	Rilascio prenotazioni scadute	11
8	Classi di progettazione	12
8.1	Classi del dominio	12
8.2	Controller	12
8.3	DAO	13
8.4	State Pattern	13
8.5	Strategy Pattern	13
9	Diagramma dei componenti	13
10	Macchine a stati	15

10.1 Macchina a stati del biglietto	15
11 Diagramma di deployment	16
12 Mockup	17
12.1 KioskInterface	17
12.2 ClerkInterface	18
13 Implementazione	18
13.1 Tecnologie	18
13.2 Controller con Dependency Injection	18
13.3 Acquisto	19
13.4 DAO	19
13.5 State Pattern	20
13.6 SubscriptionPricing	20
13.7 Concorrenza	20
14 PyUnit	21
14.1 Strategia di test	21
14.2 Test di SubscriptionPricing	21
14.3 Test del controller	22
14.4 Test dello stato	22
14.5 Copertura requisiti	23
Conclusione	23

1 Descrizione del progetto

1.1 Obiettivo generale

TicketSeller è un'applicazione per la vendita e la gestione di titoli di accesso a eventi teatrali, musicali e culturali. Il sistema permette ai clienti di consultare gli eventi disponibili, scegliere i posti, bloccarli temporaneamente durante il checkout, pagare tramite un servizio esterno simulato e ricevere il titolo di accesso acquistato.

Il sistema prevede due categorie principali di utenti:

- **Cliente:** consulta gli eventi, seleziona posti e acquista biglietti tramite interfaccia kiosk.
- **Cassiere:** gestisce il catalogo eventi tramite interfaccia amministrativa.

Il dominio comprende eventi, sale, posti, biglietti, voucher, clienti, cassieri, abbonamenti, pagamenti e politiche di prezzo. L'acquisto non è trattato come una semplice scrittura nel database, ma come una transazione applicativa composta da verifica disponibilità, blocco temporaneo, calcolo del prezzo, pagamento e conferma o annullamento della prenotazione.

1.2 Scelte architetturali

Il progetto applica principi fondamentali dell'Ingegneria del Software:

- **Separazione dei problemi:** view, controller, dominio, persistenza e servizi esterni hanno responsabilità distinte.
- **Incapsulamento:** le regole di stato del biglietto sono contenute negli oggetti stato.
- **Open/Closed Principle:** nuove politiche di prezzo possono essere aggiunte senza modificare il controller.
- **Dependency Injection:** il controller riceve DAO, servizio di pagamento e strategia di prezzo dall'esterno.
- **Testabilità:** i test PyUnit possono sostituire le dipendenze reali con mock e stub.

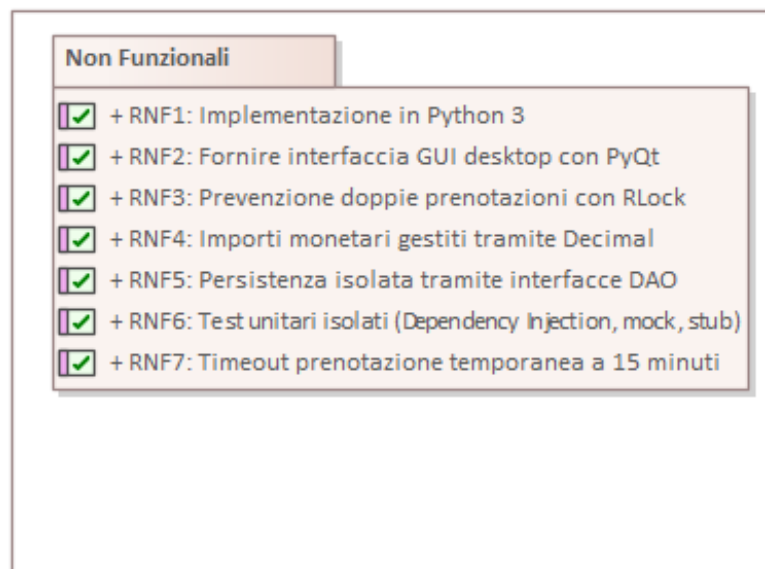
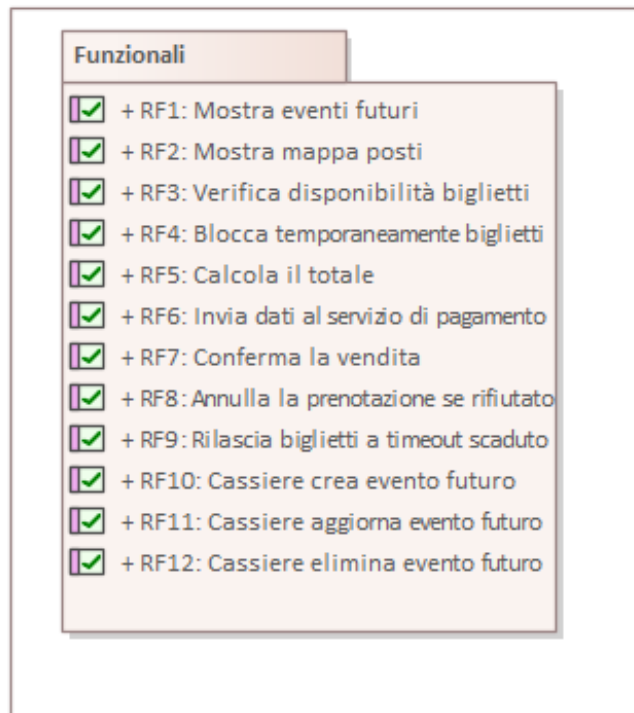
1.3 Architettura generale

TicketSeller segue una struttura Model-View-Controller integrata con un layer DAO:

Layer	Responsabilità
Model	Contiene le entità di dominio: Event , Ticket , Subscription , Customer , Clerk , Voucher .
View	Contiene KioskInterface e ClerkInterface , realizzate con PyQt.
Controller	Contiene TicketSeller , che coordina i casi d'uso.
DAO	Astrazione della persistenza tramite EventDAO e TicketDAO .
Servizi esterni	Contiene il servizio CreditCardCharges , conforme al protocollo PaymentService .

La scelta MVC/DAO concretizza la separazione dei problemi: la GUI non gestisce la persistenza, il controller non dipende dal database concreto e le regole di dominio restano nelle classi modello e nei pattern comportamentali.

2 Analisi dei requisiti



2.1 Criteri di qualità

I requisiti sono formulati secondo lo schema discusso nella teoria dell'Ingegneria dei requisiti: condizione, soggetto, azione e oggetto. Un requisito deve essere chiaro, verificabile, non ambiguo e tracciabile verso casi d'uso, classi e test.

2.2 Attori

Attore	Descrizione
Cliente	Utente finale che consulta eventi, seleziona posti e acquista biglietti.
Cassiere	Operatore autorizzato alla gestione degli eventi.
Servizio di pagamento	Sistema esterno simulato che approva o rifiuta il pagamento.
Sistema di persistenza	Componente responsabile della memorizzazione di eventi e biglietti.

2.3 Requisiti funzionali

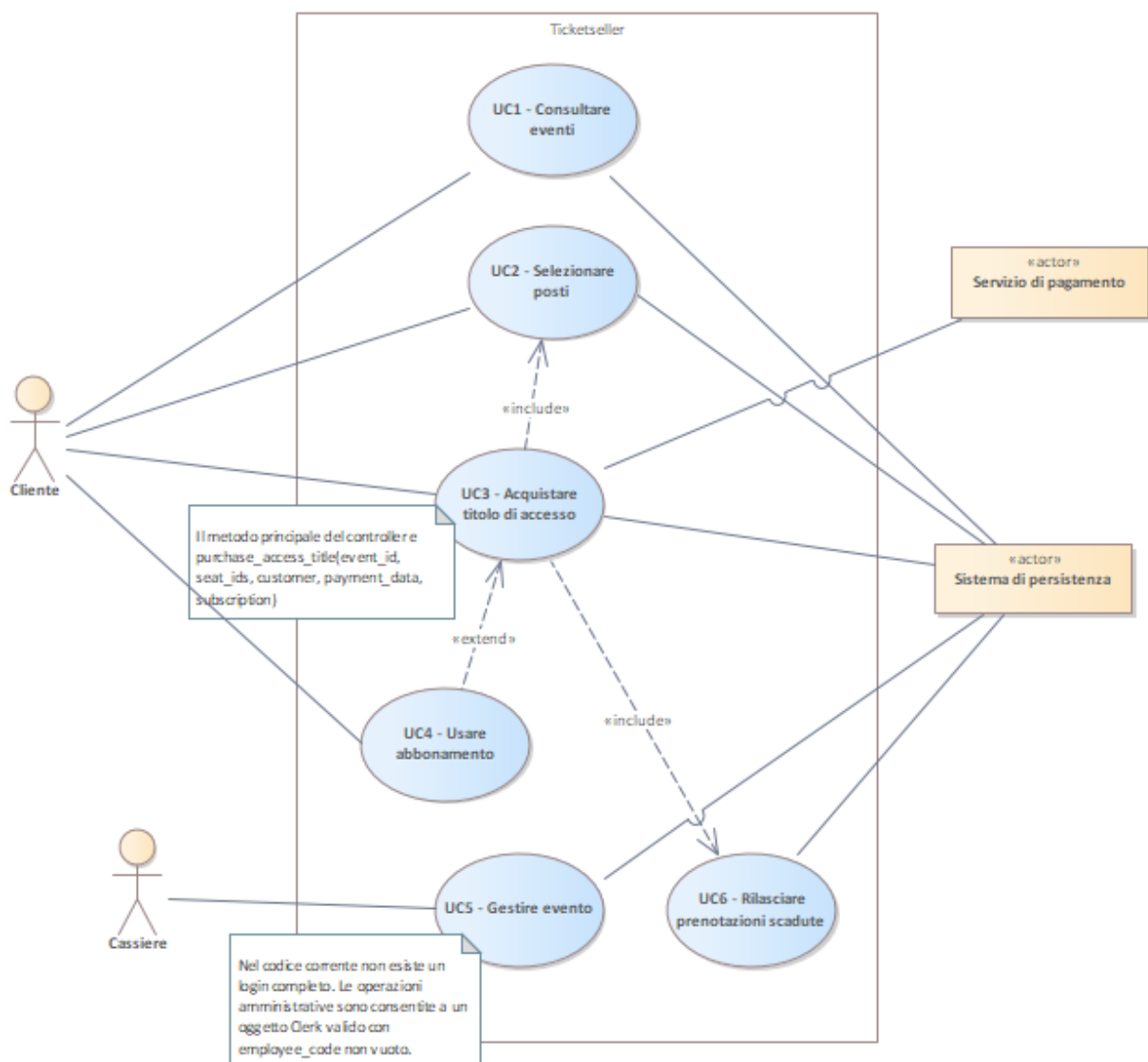
ID	Requisito
RF1	Quando il cliente accede al kiosk, il sistema deve mostrare al cliente gli eventi futuri disponibili.
RF2	Quando il cliente seleziona un evento, il sistema deve mostrare al cliente la mappa dei posti dell'evento.
RF3	Quando il cliente seleziona uno o più posti, il sistema deve verificare la disponibilità dei biglietti corrispondenti.
RF4	Quando i posti selezionati sono disponibili, il sistema deve bloccare temporaneamente i biglietti tramite <code>reserve_seats()</code> .
RF5	Quando il cliente procede al checkout, il sistema deve calcolare il totale tramite la strategia di prezzo attiva.
RF6	Quando il cliente inserisce i dati di pagamento, il sistema deve inviare la richiesta al servizio di pagamento.
RF7	Quando il pagamento è approvato, il sistema deve confermare la vendita tramite <code>confirm_sale()</code> .
RF8	Quando il pagamento è rifiutato, il sistema deve annullare la prenotazione tramite <code>cancel_reservation()</code> .
RF9	Quando una prenotazione supera il timeout, il sistema deve rilasciare i biglietti tramite <code>release_expired_reservations()</code> .
RF10	Quando il cassiere autorizzato inserisce un evento futuro, il sistema deve creare l'evento tramite <code>create_event()</code> .
RF11	Quando il cassiere autorizzato modifica un evento futuro, il sistema deve aggiornarlo tramite <code>update_event()</code> .
RF12	Quando il cassiere autorizzato richiede la rimozione di un evento, il sistema deve eliminarlo tramite <code>delete_event()</code> .
RF13	Quando il cliente possiede un abbonamento valido, il sistema deve applicare <code>SubscriptionPricing</code> ai biglietti coperti.

2.4 Requisiti non funzionali

ID	Requisito
RNF1	Il sistema deve essere implementato in Python 3.

RNF2	Il sistema deve fornire un'interfaccia grafica desktop realizzata con PyQt.
RNF3	Il sistema deve prevenire doppie prenotazioni concorrenti tramite RLock.
RNF4	Il sistema deve rappresentare gli importi monetari tramite <code>Decimal</code> .
RNF5	Il sistema deve isolare la persistenza dietro le interfacce DAO.
RNF6	Il sistema deve supportare test unitari isolati tramite Dependency Injection, mock e stub.
RNF7	Il sistema deve mantenere una prenotazione temporanea per un massimo di 15 minuti.

3 Diagramma dei casi d'uso



3.1 Casi d'uso principali

ID	Caso d'uso	Descrizione
UC1	Consultare eventi	Il cliente visualizza gli eventi futuri disponibili.
UC2	Selezionare posti	Il cliente seleziona evento e posti dalla mappa sala.
UC3	Acquistare titolo	Il cliente acquista biglietti tramite prenotazione, pagamento e conferma.
UC4	Usare abbonamento	Il cliente applica un abbonamento agli eventi coperti.
UC5	Gestire evento	Il cassiere crea, modifica o elimina eventi.
UC6	Rilasciare scadenze	Il sistema libera le prenotazioni scadute.

3.2 Caso d'uso UC3

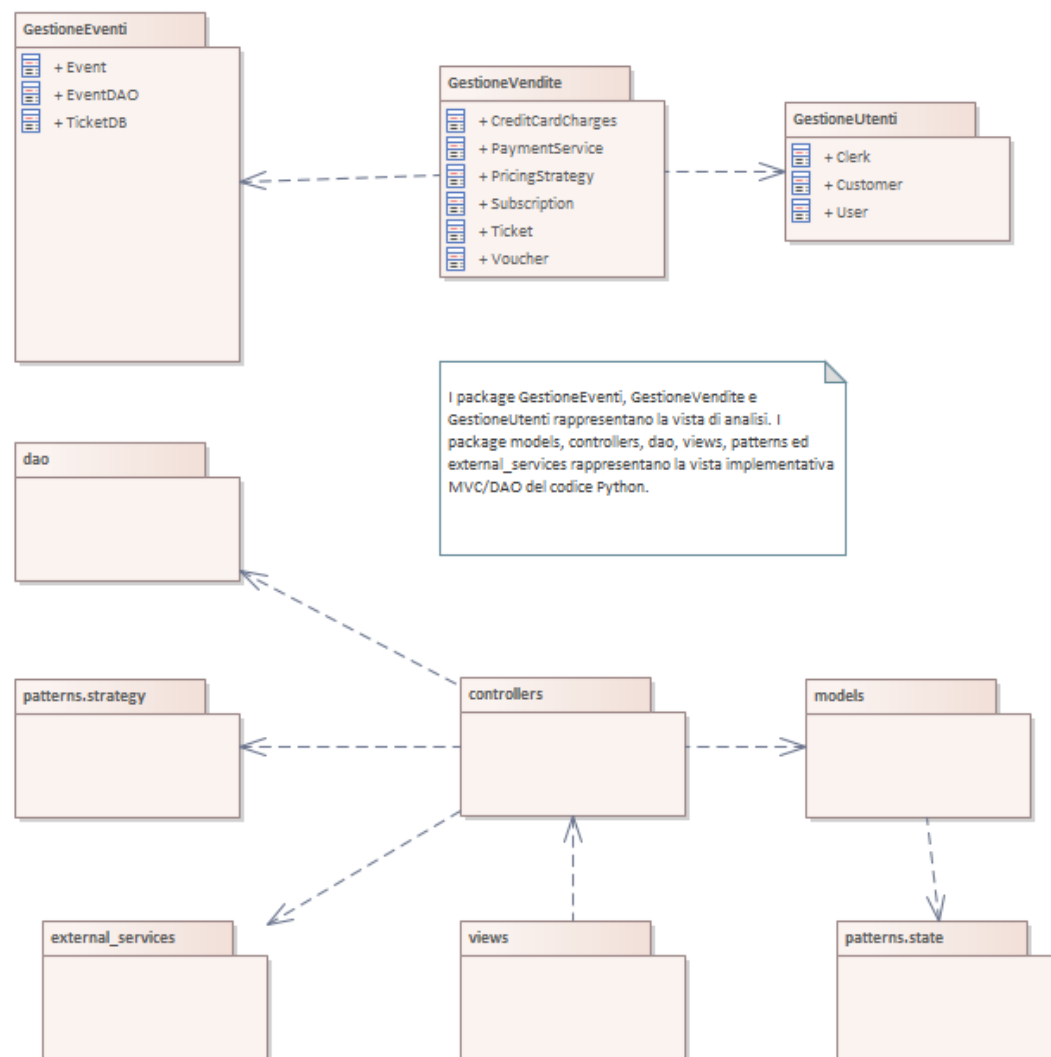
Campo	Descrizione
Nome	Acquistare titolo di accesso.
Attore primario	Cliente.
Attori secondari	Servizio di pagamento e sistema di persistenza.
Precondizioni	Il cliente ha selezionato un evento futuro e almeno un posto disponibile.
Successo	I biglietti sono in <code>SoldState</code> e associati al cliente.
Fallimento	I biglietti tornano disponibili se il pagamento viene rifiutato.
Scenario principale	La view richiama <code>purchase_access_title()</code> ; il controller prenota, calcola il totale, richiede il pagamento e conferma la vendita.
Scenario alternativo	Se il pagamento è rifiutato, il controller richiama <code>cancel_reservation()</code> .

4 Matrice di mapping

Req.	UC	Elementi progettuali	Test
RF1	UC1	<code>search_future_events()</code> , <code>find_future_events()</code>	Controller con DAO stub.
RF2	UC2	<code>get_seating_map()</code> , <code>find_tickets_by_event()</code>	Test recupero posti.
RF3	UC2	<code>check_availability()</code>	Test disponibilit�.
RF4	UC3	<code>reserve_seats()</code> , <code>Ticket.reserve()</code>	Test State Pattern.
RF5	UC3	<code>calculate_total()</code> e <code>PricingStrategy</code>	<code>test_pricing_strategy.py</code> .
RF6	UC3	<code>process_payment()</code>	Mock pagamento.
RF7	UC3	<code>confirm_sale()</code>	Test pagamento approvato.
RF8	UC3	<code>cancel_reservation()</code>	Test pagamento rifiutato.

RF9	UC6	<code>release_expired_reservations()</code>	Test timeout.
RF10	UC5	<code>create_event()</code>	Test autorizzazione.
RF11	UC5	<code>update_event()</code>	Test aggiornamento.
RF12	UC5	<code>delete_event()</code>	Test cancellazione.
RF13	UC4	SubscriptionPricing	Test abbonamento.

5 Package di analisi



5.1 Package logici

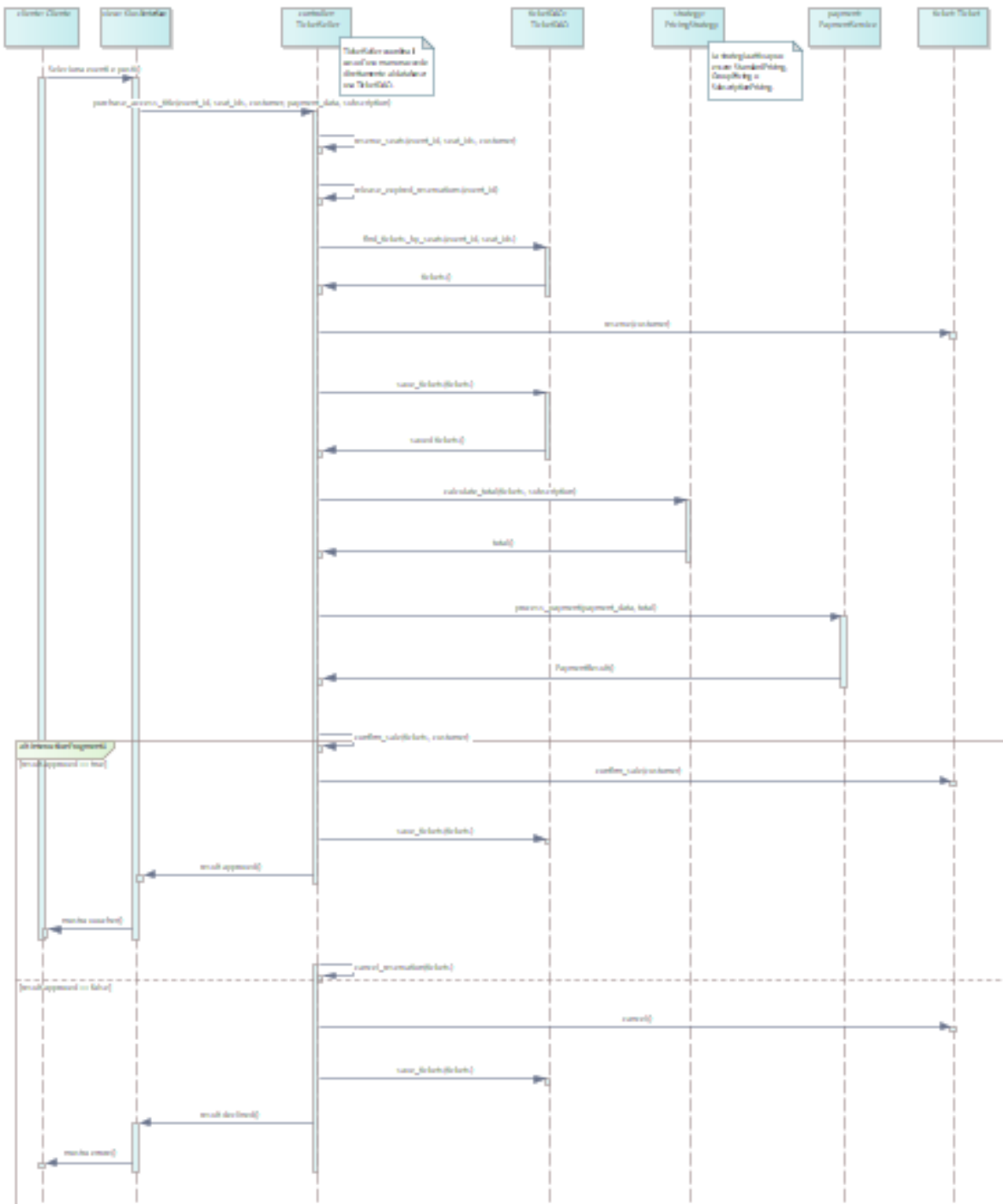
A livello di analisi il sistema è suddiviso in tre package:

- **GestioneEventi**: consultazione e amministrazione degli eventi.
- **GestioneVendite**: disponibilità, prenotazione, pagamento, conferma e annullamento.
- **GestioneUtenti**: clienti, cassieri e autorizzazioni.

5.2 Package implementativi

Package	Responsabilita
<code>models</code>	Entita di dominio.
<code>controllers</code>	Controller TicketSeller.
<code>dao</code>	Interfacce EventDAO, TicketDAO e implementazione TicketDB.
<code>external_services</code>	Servizio CreditCardCharges.
<code>patterns.state</code>	Stati del ciclo di vita del biglietto.
<code>patterns.strategy</code>	Strategie di prezzo.
<code>views</code>	Interfacce KioskInterface e ClerkInterface.

6 Diagramma di sequenza



6.1 Scenario UC3

Le lifeline principali sono Cliente, KioskInterface, TicketSeller, TicketDAO, PricingStrategy, PaymentService e Ticket.

Sequenza nominale:

1. Il Cliente seleziona evento e posti.

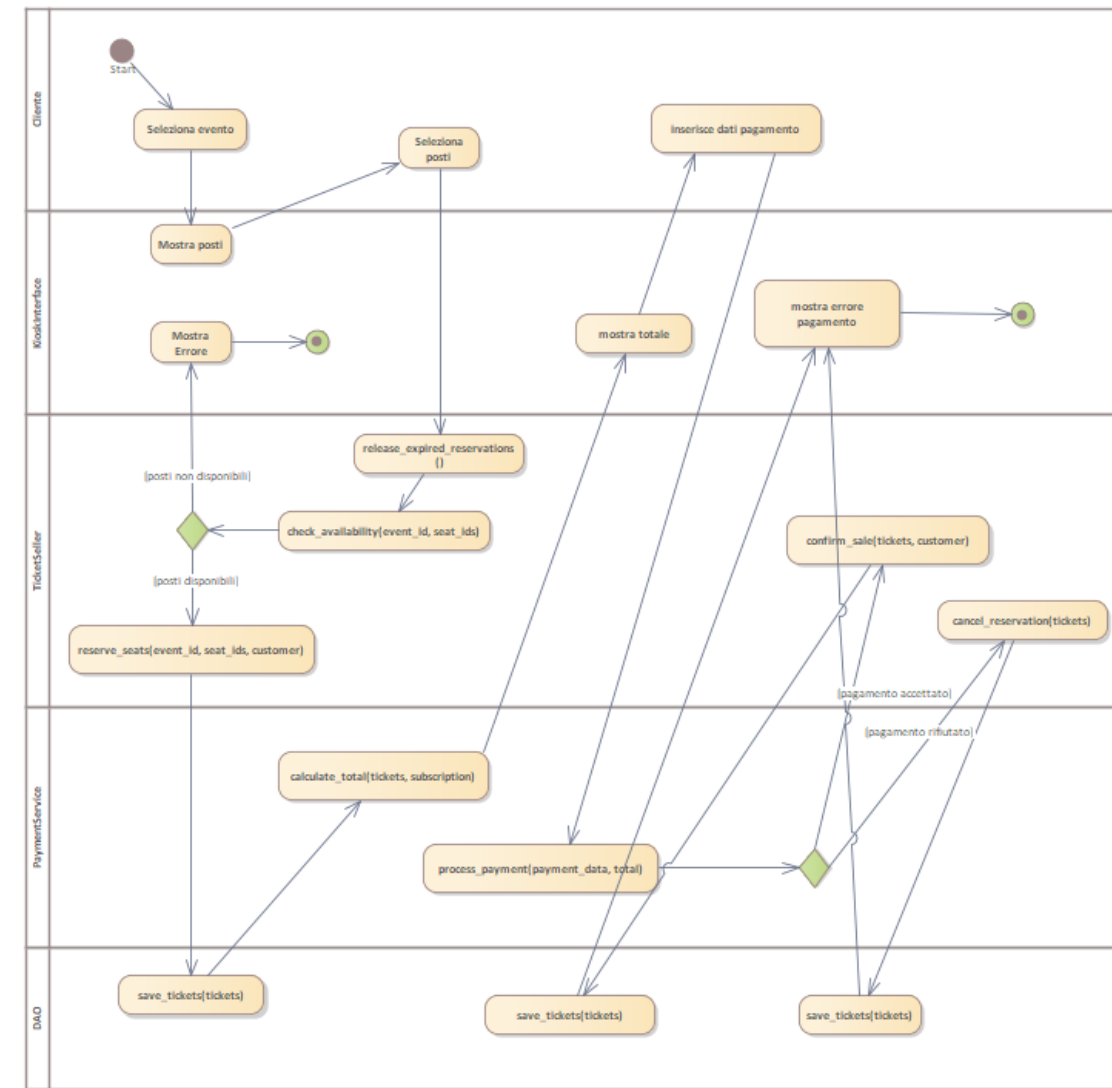
2. La view richiama `purchase_access_title()`.
3. Il controller richiama `reserve_seats()`.
4. Il controller recupera i biglietti con `find_tickets_by_seats()`.
5. Ogni biglietto disponibile riceve `reserve()`.
6. La strategia calcola il totale con `calculate_total(tickets, subscription)`.
7. Il pagamento viene richiesto tramite `process_payment()`.
8. Se approvato, il controller richiama `confirm_sale()`.
9. Se rifiutato, il controller richiama `cancel_reservation()`.
10. I biglietti aggiornati sono salvati tramite `save_tickets()`.

6.2 Scenario UC5

Il cassiere usa `ClerkInterface`. La view richiama `create_event()`, `update_event()` o `delete_event()`. Il controller verifica il cassiere e delega a `EventDAO`.

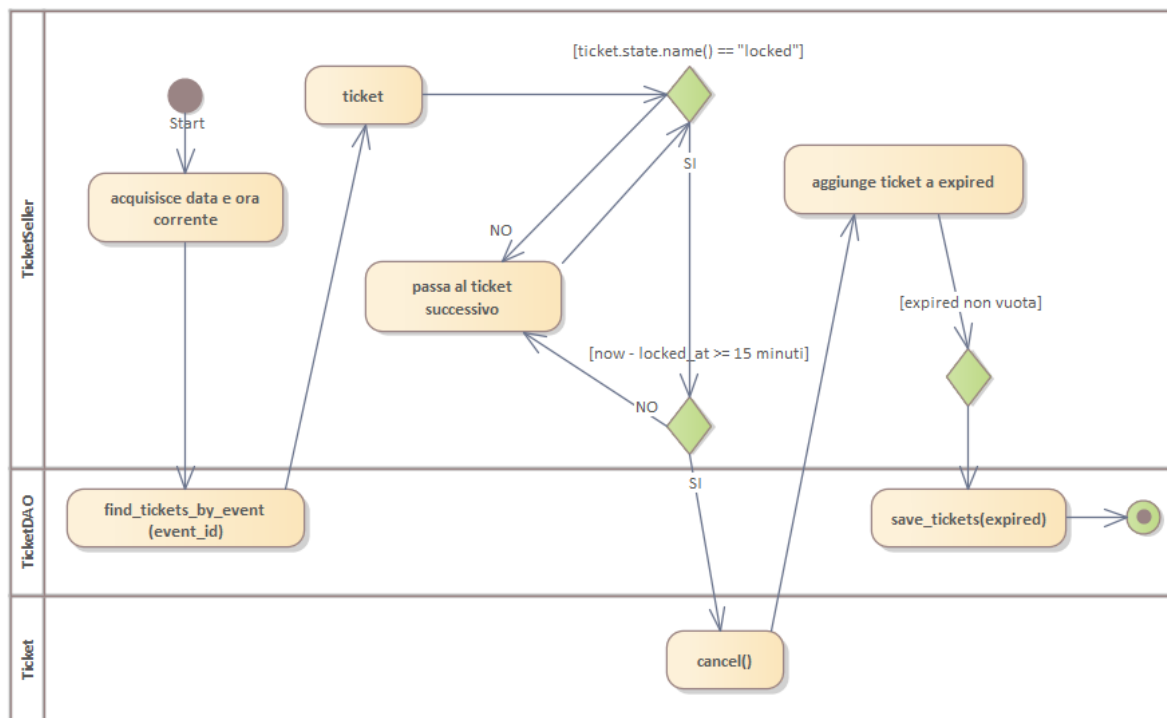
7 Diagrammi di attivita

7.1 Acquisto biglietto



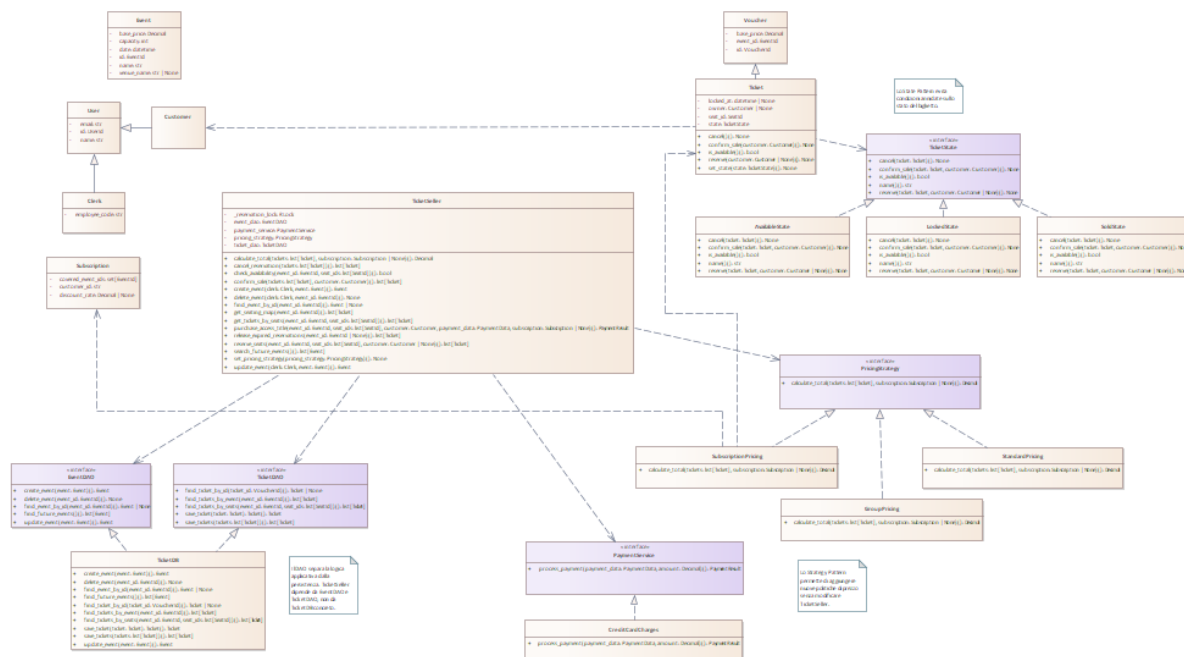
1. Il cliente sceglie un evento.
2. Il sistema mostra la mappa dei posti.
3. Il cliente seleziona i posti.
4. Il sistema verifica la disponibilit .
5. Se i posti non sono disponibili, mostra errore.
6. Se i posti sono disponibili, esegue `reserve_seats()`.
7. Il sistema calcola il totale.
8. Il cliente inserisce i dati di pagamento.
9. Il sistema invia la richiesta al servizio di pagamento.
10. Se il pagamento   approvato, esegue `confirm_sale()`.
11. Se il pagamento   rifiutato, esegue `cancel_reservation()`.

7.2 Rilascio prenotazioni scadute



1. Il sistema recupera l'orario corrente.
2. Il sistema recupera i biglietti da controllare.
3. Per ogni biglietto verifica se lo stato è **locked**.
4. Se il timeout di 15 minuti è superato, richiama **cancel()**.
5. I biglietti rilasciati sono salvati tramite **save_tickets()**.

8 Classi di progettazione



8.1 Classi del dominio

Classe	Responsabilita
Event	Rappresenta un evento vendibile.
Voucher	Classe base dei titoli di accesso.
Ticket	Rappresenta un biglietto associato a un posto.
Subscription	Rappresenta un abbonamento con eventi coperti e sconto.
Customer	Rappresenta il cliente.
Clerk	Rappresenta il cassiere autorizzato.

8.2 Controller

`TicketSeller` coordina i casi d'uso. I metodi principali sono:

- `check_availability(event_id, seat_ids)`
- `reserve_seats(event_id, seat_ids, customer)`
- `purchase_access_title(event_id, seat_ids, customer, payment_data, subscription)`
- `calculate_total(tickets, subscription)`
- `confirm_sale(tickets, customer)`
- `cancel_reservation(tickets)`
- `release_expired_reservations(event_id)`

- `create_event(clerk,event)`
- `update_event(clerk,event)`
- `delete_event(clerk,event_id)`

8.3 DAO

Il Data Access Object separa la logica applicativa dalla persistenza. Le interfacce principali sono `EventDAO` e `TicketDAO`; `TicketDB` le implementa entrambe. Il controller dipende dalle astrazioni, non dal database concreto.

8.4 State Pattern

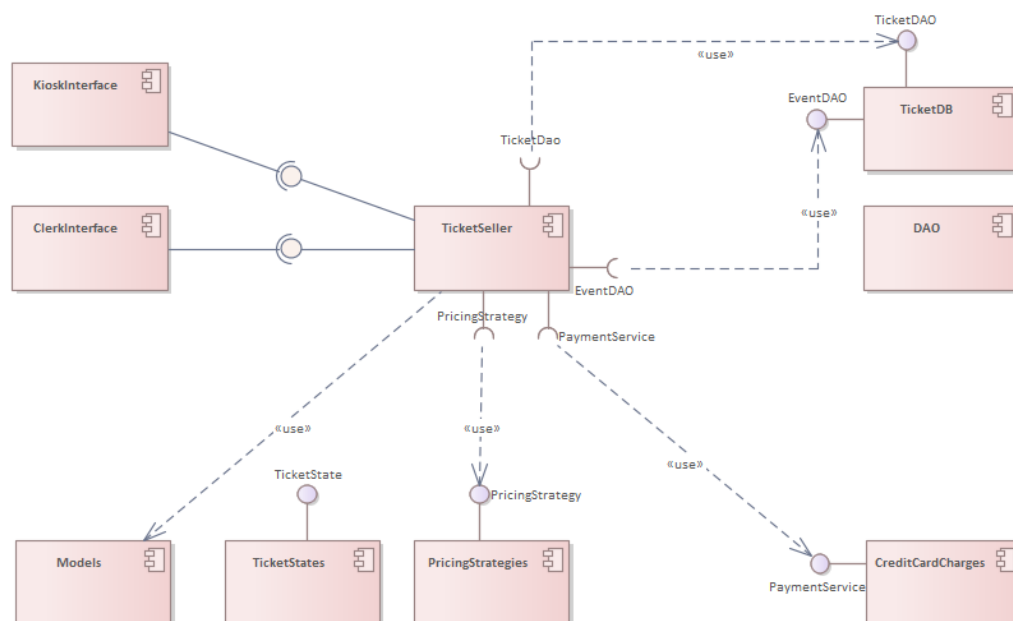
Gli stati sono `AvailableState`, `LockedState` e `SoldState`. Lo State Pattern evita istruzioni condizionali complesse nel controller e nella classe `Ticket`; ogni stato conosce le operazioni consentite.

8.5 Strategy Pattern

Il prezzo è calcolato tramite `PricingStrategy`. Le implementazioni sono `StandardPricing`, `GroupPricing` e `SubscriptionPricing`. La firma reale è `calculate_total(tickets,subscription)`.

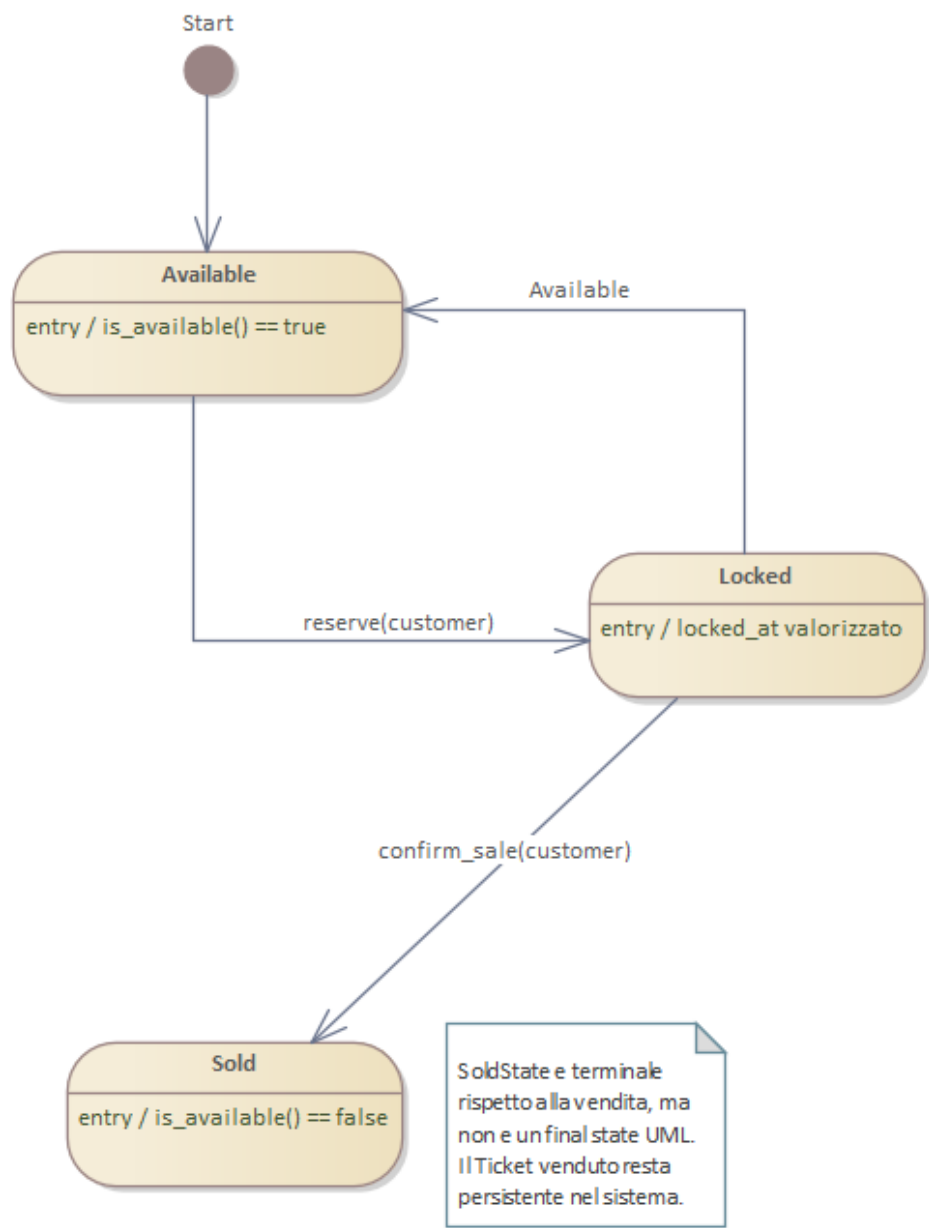
`SubscriptionPricing` itera sulla lista di `Ticket`: se l'evento del ticket è incluso in `subscription.covered_event_ids`, applica lo sconto dell'abbonamento; altrimenti somma il prezzo pieno. Questa soluzione rispetta l'Open/Closed Principle perché nuove politiche di prezzo possono essere aggiunte senza modificare `TicketSeller`.

9 Diagramma dei componenti



Componente	Descrizione
KioskInterface	Presentazione per il cliente.
ClerkInterface	Presentazione per il cassiere.
TicketSeller	Controller applicativo.
Models	Entita di dominio.
DAO	Interfacce di persistenza.
TicketDB	Implementazione della persistenza.
PricingStrategies	Politiche di prezzo.
TicketStates	Stati del biglietto.
CreditCardCharges	Servizio di pagamento simulato.

10 Macchine a stati

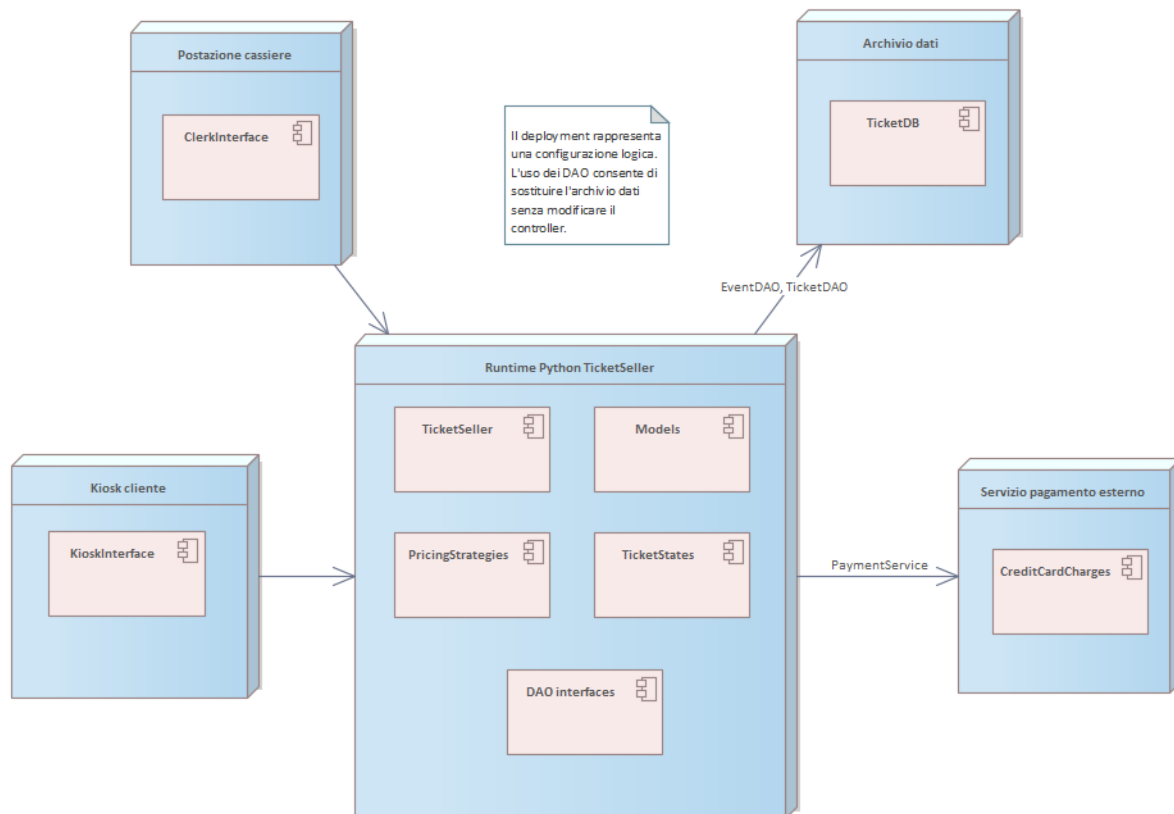


10.1 Macchina a stati del biglietto

Da	A	Evento	Effetto
Available	Locked	<code>reserve()</code>	Blocca temporaneamente il biglietto.
Locked	Available	<code>cancel()</code>	Rilascia il posto.
Locked	Sold	<code>confirm_sale()</code>	Conferma la vendita.
Sold	Sold	Operazioni non valide	Il biglietto resta venduto.

SoldState e terminale rispetto al processo di vendita, ma non deve essere modellato come final state UML. L'oggetto **Ticket** venduto continua a esistere nel sistema come titolo persistente.

11 Diagramma di deployment



Nodo	Componenti
Kiosk	KioskInterface, runtime Python.
Postazione cassiere	ClerkInterface, runtime Python.
Applicazione	TicketSeller, modelli, stati e strategie.
Archivio dati	TicketDB o altro DAO compatibile.
Servizio pagamento	CreditCardCharges o adattatore compatibile.

12 Mockup

12.1 KioskInterface

The screenshot displays the 'TicketSeller Kiosk' application window. It features a 'Select event' dropdown menu currently showing 'Jazz Night (2026-08-23 16:04)'. Below this is a 'Select one or more available seats' section with a list of six seats: S001, S002, S003, S004 (highlighted), S005, and S006, each priced at 30.00. The 'Total: 30.00' is displayed. The 'Payment' section contains input fields for Name (Demo Customer), Email (customer@example.com), Card (4111111111111111), CVV (123), Exp. Month (12), and Exp. Year (2099). At the bottom, there are 'Refresh' and 'Buy Selected Seats' buttons.

TicketSeller Kiosk

Select event

Jazz Night (2026-08-23 16:04)

Select one or more available seats

S001 - available - 30.00

S002 - available - 30.00

S003 - available - 30.00

S004 - available - 30.00

S005 - available - 30.00

S006 - available - 30.00

Total: 30.00

Payment

Name Demo Customer

Email customer@example.com

Card 4111111111111111

CVV 123

Exp. Month 12

Exp. Year 2099

Refresh Buy Selected Seats

Le schermate principali sono: elenco eventi, dettaglio evento, mappa posti, riepilogo ordine, pagamento e voucher o errore.

12.2 ClerkInterface

The screenshot shows a desktop application window titled "TicketSeller Clerk". Inside the window, there is a section labeled "Events" containing a list of two items:

- 1 - Jazz Night - 2026-08-23 16:05
- 2 - Classical Gala - 2026-08-30 16:05

Below the list, there is a form with several input fields and buttons:

- ID**: A text box containing the value "1".
- Name**: A text box containing the value "Jazz Night".
- Date**: A date and time picker showing "23/08/2026 16:05".
- Capacity**: A text box containing the value "20".
- Base Price**: A text box containing the value "30.00".
- Venue**: A text box containing the value "Blue Hall".

At the bottom of the form, there are four buttons arranged vertically:

- Create
- Update
- Delete
- Refresh

Le schermate principali sono: elenco eventi, creazione evento, modifica evento, eliminazione evento e messaggi di validazione.

13 Implementazione

13.1 Tecnologie

Il sistema è implementato in Python 3. Le viste sono realizzate con PyQt, gli importi con `Decimal`, i test con `PyUnit`. Il progetto separa modelli, controller, DAO, servizi esterni, pattern e viste.

13.2 Controller con Dependency Injection

```
class TicketSeller:
    reservation_timeout = timedelta(minutes=15)

    def __init__(
        self,
        event_dao: EventDAO,
        ticket_dao: TicketDAO,
```

```

        payment_service: PaymentService,
        pricing_strategy: PricingStrategy,
    ) -> None:
        self.event_dao = event_dao
        self.ticket_dao = ticket_dao
        self.payment_service = payment_service
        self.pricing_strategy = pricing_strategy
        self._reservation_lock = RLock()

```

13.3 Acquisto

```

def purchase_access_title(
    self,
    event_id: EventId,
    seat_ids: list[SeatId],
    customer: Customer,
    payment_data: PaymentData,
    subscription: Subscription | None = None,
) -> PaymentResult:
    tickets = self.reserve_seats(event_id, seat_ids, customer)
    total = self.calculate_total(tickets, subscription)
    result = self.payment_service.process_payment(payment_data, total)

    if result.approved:
        self.confirm_sale(tickets, customer)
    else:
        self.cancel_reservation(tickets)

    return result

```

13.4 DAO

```

class EventDAO(ABC):
    @abstractmethod
    def find_future_events(self) -> list[Event]:
        pass

    @abstractmethod
    def find_event_by_id(self, event_id: EventId) -> Event | None:
        pass

    @abstractmethod
    def create_event(self, event: Event) -> Event:
        pass

class TicketDAO(ABC):
    @abstractmethod
    def find_tickets_by_event(self, event_id: EventId) -> list[Ticket]:
        pass

    @abstractmethod
    def find_tickets_by_seats(
        self,
        event_id: EventId,
        seat_ids: list[SeatId],
    ) -> list[Ticket]:
        pass

```

```
@abstractmethod
def save_tickets(self, tickets: list[Ticket]) -> list[Ticket]:
    pass
```

13.5 State Pattern

```
class Ticket(Voucher):
    def reserve(self, customer: Customer | None = None) -> None:
        self.state.reserve(self, customer)

    def cancel(self) -> None:
        self.state.cancel(self)

    def confirm_sale(self, customer: Customer) -> None:
        self.state.confirm_sale(self, customer)

    def is_available(self) -> bool:
        return self.state.is_available()
```

13.6 SubscriptionPricing

```
class SubscriptionPricing(PricingStrategy):
    def calculate_total(
        self,
        tickets: list[Ticket],
        subscription: Subscription | None = None,
    ) -> Decimal:
        if subscription is None:
            raise ValueError("Subscription required.")

        discount_rate = subscription.discount_rate
        if discount_rate is None:
            discount_rate = Decimal("1.00")

        total = Decimal("0.00")

        for ticket in tickets:
            if ticket.event_id in subscription.covered_event_ids:
                total += ticket.base_price * (
                    Decimal("1.00") - discount_rate
                )
            else:
                total += ticket.base_price

        return quantize_money(total)
```

13.7 Concorrenza

```
def reserve_seats(
    self,
    event_id: EventId,
    seat_ids: list[SeatId],
    customer: Customer | None = None,
) -> list[Ticket]:
    with self._reservation_lock:
```

```
self.release_expired_reservations(event_id)

if len({str(seat_id) for seat_id in seat_ids}) != len(seat_ids):
    raise ValueError("Duplicate seats cannot be selected.")

tickets = self.ticket_dao.find_tickets_by_seats(event_id, seat_ids)

if not tickets:
    raise ValueError("At least one seat must be selected.")

if any(not ticket.is_available() for ticket in tickets):
    raise ValueError("One or more seats are not available.")

for ticket in tickets:
    ticket.reserve(customer)

return self.ticket_dao.save_tickets(tickets)
```

14 PyUnit

14.1 Strategia di test

I test sono organizzati per responsabilit :

- `test_pricing_strategy.py`: strategie di prezzo.
- `test_ticket_state.py`: transizioni di stato.
- `test_ticket_seller_controller.py`: controller.
- `test_ticket_db.py`: persistenza.
- `test_payment_service.py`: pagamento.
- `test_model_validation.py`: validazione modelli.
- `test_utils.py`: funzioni di utilita.

I test usano mocking, stubbing e Dependency Injection. Questo   coerente con la teoria di verifica e validazione: il test unitario deve isolare l'unit  sotto test e controllare in modo deterministico le dipendenze esterne.

14.2 Test di SubscriptionPricing

```
def test_subscription_pricing_only_covered_events(self):
    tickets = [
        Ticket(
            id=VoucherId("T1"),
            event_id=EventId("E1"),
            base_price=Decimal("100.00"),
            seat_id=SeatId("A1"),
        ),
        Ticket(
            id=VoucherId("T2"),
            event_id=EventId("E2"),
            base_price=Decimal("80.00"),
            seat_id=SeatId("A2"),
```

```

    ),
]

subscription = Subscription(
    id="S1",
    customer_id="C1",
    covered_event_ids={EventId("E1")},
    discount_rate=Decimal("1.00"),
)

total = SubscriptionPricing().calculate_total(
    tickets,
    subscription,
)

self.assertEqual(total, Decimal("80.00"))

```

14.3 Test del controller

```

def test_purchase_confirms_sale_when_payment_is_approved(self):
    payment_service = Mock()
    payment_service.process_payment.return_value = PaymentResult(
        approved=True,
        transaction_id="TX-1",
        message="approved",
    )

    controller = TicketSeller(
        event_dao=event_dao_stub,
        ticket_dao=ticket_dao_stub,
        payment_service=payment_service,
        pricing_strategy=StandardPricing(),
    )

    result = controller.purchase_access_title(
        event_id=EventId("E1"),
        seat_ids=[SeatId("A1")],
        customer=customer,
        payment_data=payment_data,
    )

    self.assertTrue(result.approved)
    self.assertEqual(ticket.state.name(), "sold")

```

14.4 Test dello stato

```

def test_locked_ticket_can_be_confirmed_as_sold(self):
    ticket = Ticket(
        id=VoucherId("T1"),
        event_id=EventId("E1"),
        base_price=Decimal("50.00"),
        seat_id=SeatId("A1"),
    )

    ticket.reserve(customer)
    ticket.confirm_sale(customer)

```

```
self.assertEqual(ticket.state.name(), "sold")
self.assertFalse(ticket.is_available())
```

14.5 Copertura requisiti

Requisito	Comportamento testato	Tecnica
RF1, RF2	Eventi e mappa posti	Stub DAO.
RF3, RF4	Disponibilit� e prenotazione	Test controller e State.
RF5, RF13	Calcolo totale	Test Strategy.
RF6	Pagamento	Mock <code>PaymentService</code> .
RF7	Conferma vendita	Mock pagamento approvato.
RF8	Annullamento	Mock pagamento rifiutato.
RF9	Timeout	Fixture con <code>locked_at</code> scaduto.
RF10–RF12	CRUD eventi	Stub <code>EventDAO</code> .

Conclusione

TicketSeller applica in modo integrato analisi dei requisiti, modellazione UML, progettazione orientata agli oggetti, pattern architetturali e comportamentali, implementazione Python e testing unitario. L'architettura MVC separa presentazione, controllo e dominio; il layer DAO isola la persistenza; lo State Pattern modella il ciclo di vita del biglietto; lo Strategy Pattern rende estendibili le politiche di prezzo; la Dependency Injection migliora testabilit  e manutenibilit .

La documentazione   allineata al codice reale, inclusi i metodi `reserve()`, `reserve_seats()`, `purchase_access_title()`, `confirm_sale()`, `cancel_reservation()` e `release_expired_reservations()`.